

2.6 语音交互

1. 什么是语音交互

语音交互（Voice Interaction, VI）是指人类与计算机或设备之间通过语音进行交流的一种交互方式。语音交互结合了语音识别和语音合成，使得设备能够听懂用户的需求并以自然的语音回应，实现双向语音交流。要实现自然的语音交互，需考虑语义理解、情感分析等因素，以便系统能够准确判断用户意图并作出合理回应。

我们可以根据此玩法实现我们的 AI 小助手功能。

2. 实现原理

首先，通过唤醒检测模块监听特定的唤醒词，触发后开始录音。录音后，自动语音识别（ASR）将音频转换为文本，随后将文本传递给大语言模型（LLM）生成相应的回答。生成的文本会通过文本到语音转换（TTS）模块转化为语音，并播放给用户。整个流程实现了用户与语音助手的自然互动。

3. 实验前准备

- 1) 需要准备网线或者连接 Wi-Fi 进行联网，确保主控设备能正常访问网络。
- 2) 需要参考“1 大语言模型课程/1.2 大语言模型部署”课程获取“通义千问”及“阶跃星辰”密钥并参考“1 大语言模型课程/1.3 大语言模型调用”将对应密钥填进“`confing.py`”里。

4. 实验步骤

- 1) 将设备开机，并通过 MobaXterm 连接（连接方法可参照附录的“5.1 远程

连接工具及说明”内容）。

2) 我们需要查询我们接入的 WonderEchoPro/环形麦克风（后面统称为**语音设备**）的 USB 设备号名字，具体的步骤如下（**注意**：查询的过程不要接入其他的 USB 设备）。

- 首先**断开**语音设备，然后在命令行终端输入以下指令。

```
ls /dev | grep USB
```

```
> ls /dev | grep USB  
ttyCH341USB0
```

- 接着我们将语音设备**接入**到我们的主板的 USB 接口上，然后输入指令，查看当前主板的 USB 设备名字。

```
ls /dev | grep USB
```

```
> ls /dev | grep USB
```

- 可以看到接入语音设备之后，出现了一个 USB 端口为 **ttyCH341USB1**（需要记住这个设备名字），不同的主控输出名字可能会有不同。

```
> ls /dev | grep USB  
ttyCH341USB0  
ttyCH341USB1
```

- 3) 输入指令到程序目录下：

```
cd large_models/
```

```
cd large_models/
```

4) 输入指令,打开程序填写自己的 API Key 和修改完后按下 Esc 键,输入“:wq”,并按下回车,即可保存退出程序。

```
vim config.py
```

```
4  
5 ###Chinese  
6 api_key = ' '   
7 base_url = 'https://dashscope.aliyuncs.com/compatible-mode/v1'  
8 dashscope.api_key = api_key  
9
```

5) 填入查询到的端口编号，修改对应的麦克风端口 WonderEcho Pro 或者六路麦克风，解开要使用端口的注释，注释掉未使用端口的设置。

```
vim interaciton_demo.py
```

WonderEcho Pro 修改为：

```
15 port = '/dev/ttyCH341USB1'
16 kws = awake.WonderEchoPro(port)
17 # kws = awake.CircleMic(port)
```

六路麦克风修改为：

```
15 port = '/dev/ttyCH341USB1'
16 #kws = awake.WonderEchoPro(port)
17 kws = awake.CircleMic(port)
18
```

6) 输入指令执行程序：

```
python3 interaciton_demo.py
```

```
> python3 interaciton_demo.py
```

7) 如需关闭此程序，可按下“Ctrl+c”即可。

5. 功能实现

成功执行后，语音设备会播报“准备就绪”，然后对其进行唤醒词“hello hiwonder”唤醒，当语音设备播报“我在”即说明成功将智能体唤醒。可以随意进行提问做我们的 ai 小助手：

例：今天天气怎么样？

```
Recording.....
今天，
今天，
今天天气怎？
今天天气怎么样？
RecognitionCallback sentence end, request_id:a1b1c0c33f9042b283ad14b69ff52ba2, usage: {'duration': 1}
RecognitionCallback completed.
Done recording
asr_result: 今天天气怎么样？
llm_response: 您当前询问的是2025年4月24日星期四的天气。不过，我所掌握的信息中没有直接提供这一天的具体天气情况。根据之前提供的信息，我可以告诉您上海在4月26日（即后天）的天气预报是多云到阴，气温在13°至22°之间，偏东风风力3-4级。
如果您想了解今天的天气情况，建议查看当地最新的气象报告或使用实时天气服务来获取最准确的信息。或者，请告诉我您所在的城市，我可以尝试为您提供更具体的帮助。
```

6. 程序简要分析

程序集成语音识别、语音合成和智能响应的语音助手，利用唤醒词（hello hiw onder）进行交互。可通过语音指令与助手对话，助手则使用文本到语音技术回答用户。整体结构清晰，各模块分工明确，易于扩展和维护。

该程序的源代码位于：/home/ubuntu/large_models/interaciton_demo.py

6.1 导入必要的模块

```
5 import os
6 import time
7 from config import *
8 from speech import awake
9 from speech import speech
```

time: 用于控制程序执行间隔。

speech: 核心模块，集成了唤醒词检测、语音活动检测、语音识别、TTS 和 LLM。

6.2 音频文件路径定义

```
11 wakeup_audio_path = './resources/audio/wakeup.wav'
12 start_audio_path = './resources/audio/start_audio.wav'
13 no_voice_audio_path = './resources/audio/no_voice.wav'
```

配置了各功能模块使用的音频文件路径，例如唤醒音、录音存储路径、提示音等。

初始化文字转语音模块，用于将 LLM 回复转换为语音。

6.3 主功能逻辑

```
31
32 def main():
33     kws.start()
34     while True:
35         try:
36             if kws.wakeup(): # 检测到唤醒词
37                 speech.play_audio(wakeup_audio_path) # 唤醒播放
38                 asr_result = asr.asr() # 开启录音识别
39                 print('asr_result:', asr_result)
40                 if asr_result:
41                     # 将识别结果传给智能体让他来回答
42                     params = {"Model": 'qwen-max',
43                               "messages": [
44                                   {
45                                       "role": "user",
46                                       "content": asr_result
47                                   },
48                               ],
49                               "extra_body": {
50                                   "enable_search": True
51                               }
52                     }
53                     response = client.llm_origin(params).choices[0].message.content.strip()
54                     print('llm response:', response)
55                     tts.tts(response)
56                 else:
57                     speech.play_audio(no_voice_audio_path)
58             else:
59                 time.sleep(0.02)
60         except KeyboardInterrupt:
61             kws.exit()
62             try:
63                 os.system('pinctrl FAN_PWM a0')
64             except:
65                 pass
66             break
67         except BaseException as e:
68             print(e)
69
```

唤醒检测：持续检测唤醒词，触发后停止唤醒检测并播放唤醒提示音。

语音处理：录音、识别用户语音，使用语言模型生成回复，并通过语音合成播放回复。

异常处理：捕获退出信号和运行错误，确保程序安全退出并释放资源。